

Computer Security Exam

Professors S. Zanero & M. Carminati & A. Barengi

Last (family) Name _____
First (given) Name _____
Matricola _____
Codice Persona _____

Professor: [] Zanero [] Carminati [] Barengi
Have you done any challenges/homework? [] Yes [] No
Do you want to withdraw? [] Yes [] No
Priority Grading [] Yes [] No
Motivation:

Instructions

- **Duration:** 2.5 hours.
- The exam is composed of 17 pages. Check that you have all of them. No extra paper is allowed.
- The exam is "closed book." You are not allowed to consult any **course material** and **notes**.
- **No extra devices** (e.g., phones, iPad) are **allowed**. You will be expelled if, at any time, you do not follow this rule.
- Shut down and store any electronic device. They will be subject to inspection if found, and you may be expelled if you are found using one.
- You are not allowed to communicate with other students, and you will be expelled from the exam if you do.
- Please answer within the allowed space. Schemes are good; short answers are recommended.
- All responses must be written in pen (any color except red). Answers written in pencil may not be legible when scanned and, therefore, may not be graded.
- **Always motivate your answer.**
- **If your final grade is below 12 (not considering the challenges), you will not be allowed to take the next exam call (even in the next session).**

DISTANCE MODE ONLY

- On the PC, you can open only the forms browser tab and the exam pdf. Any other application/document is prohibited.
- You will have to share your screen, and leave your microphone opened, and your webcam must frame both you and your workspace.
- You are responsible for having all the necessary technological equipment for the correct exam delivery.
- If you disable the screen sharing or the audio, you will be expelled from the virtual room, and your exam will be voided.
- We will ask remote students to sustain a brief oral exam to confirm the evaluation of the written exams. This could be done for all students or a subset at the instructor's decision.
- Regarding the submission:
 - At the end of the exam, you will be given 10 minutes only to scan your documents and prepare to upload them.
 - After 10 minutes, you will be called by name and required to click on the submit button of the MS forms under the supervision of the supervisor of the virtual room. Any submission that will not follow this procedure (e.g., you have already submitted before being called or you are not ready to submit when called) will be voided.
 - We are not responsible for any technical issue in the submission (e.g., the exam pictures are not readable, the pdf is corrupted). We will evaluate only the uploaded readable documents.
 - We will evaluate only the uploaded **readable** documents.
- USE YOUR PERSONAL CODE (CODICE PERSONA) ONLY AS THE NAME OF THE DOCUMENT (e.g., 12345678.pdf).

PLEASE, USE THE FORMATTING SPECIFIED IN THE QUESTION

READ CAREFULLY ALL THE POINTS OF EACH QUESTION BEFORE WRITING YOUR ANSWER

Exercise 1 - Introduction to Computer Security (4 points)

Modern ride-sharing applications (e.g., Uber, Lyft) connect passengers with drivers through a mobile app, manage ride booking, provide real-time GPS tracking for both parties, process payments automatically, and maintain a reputation system through user ratings.

The platform relies on a central cloud infrastructure where user and driver records are kept together, including personal profiles, travel history, contact information, and payment details. To ensure compatibility with a wide range of devices, some services continue to support TLS 1.0/1.1 for connections from legacy smartphones, alongside current standards. Certain backend components are still tied to Java 8 libraries to avoid breaking legacy integrations. Server and database configuration files are centrally maintained by administrators to streamline updates.

The mobile application requests a broad set of device permissions, such as access to location services, contacts, notifications, microphone, and camera. Ride coordination requires frequent exchanges between the app and backend through REST APIs, which are also documented for partner integrations. While communications are encrypted, the system retains the ability to fall back on SHA-1 certificates when stronger options are not available. Continuous connectivity is required for booking, tracking, and payment functions.

1. [1.5 points] Focusing just Identify two primary assets at risk in this scenario. For each asset, describe a related threat and a potential threat agent.

Asset	Threat	Threat Agent
User Personal & Location Data	Unauthorized access to and exfiltration of user trip history, home addresses, and payment information, leading to stalking or financial fraud.	Cybercriminals seeking to sell personal data on the dark web or use it for identity theft.
Service Availability & Integrity	A Denial of Service (DoS) attack on the platform's backend servers, preventing users from booking rides and disrupting the service for both riders and drivers.	Competitors or Hacktivists aiming to cause financial and reputational damage to the company.

2. [1.5 points] Suggest two distinct attack surfaces for the ride-sharing platform.

Attack Surface	Motivation
The Mobile Application	The app itself can contain vulnerabilities (e.g., insecure data storage, flawed API implementation) that could be exploited through reverse engineering or by tricking a user into installing a malicious version.
The Cloud Backend API	The APIs that the mobile apps use to communicate with the central servers are a critical attack surface. If not properly secured, they could be vulnerable to attacks like unauthorized data access, injection attacks, or insecure direct object references.

3. [1 point] Based on the description, identify two potential vulnerabilities that could be exploited. Explain why they represent security risks.

Vulnerability	Motivation
Support for old software and protocols for legacy support (Java 8, TLS, SHA-1)	Outdated cryptographic protocols are vulnerable to downgrade attacks and could allow attackers to intercept or tamper with communications.
Excessive Mobile App Permissions (camera/microphone not required for core service)	Creates unnecessary exposure of sensitive data (audio, images) that could be abused by malicious actors if the app or device is compromised.

(Other valid answers students might raise: centrally maintained configuration files as a single point of failure, or shared databases increasing risk of unauthorized access.)

Exercise 2 - Introduction to Cryptography (6 points)

1. [4 points] A symmetric encryption system needs to be employing 128 bit user-provided keys and AES-128 for legacy reasons. Willing to allow users to use 256 bit user-provided keys too, the developer chose this strategy: encrypt the three times with AES-128 in counter-mode. To do this the developer proceeds in this way, regardless of the user-provided key length:

- Split the key into two equally long parts, k_1 , k_2
- Compute two 128 bit strings k'_1 , k'_2 truncating to 128 bit SHA-3-256($k_1 \parallel \text{"FIRST"}$) and SHA-3-256($k_2 \parallel \text{"SECOND"}$), where $\parallel$ indicates string concatenation.
- Encrypt the plaintext p drawing a fresh, unpredictable random nonce $nonce$ and computing AES-128-CTR($nonce$, AES-128-CTR($nonce$, AES-128-CTR($nonce$, p , k'_1), k'_2), k'_1);

The triple encryption technique is specifically designed to better blend the effect of the encryption With the two key halves.

Find improvements in this approach, or prove its optimality in terms of confidentiality providing and efficiency. Integrity and endpoint authentication are not required in this scenario.

The approach breaks the confidentiality of legacy-key encrypted plaintexts: indeed, the two encryptions with the same nonce and k'_1 as key cancel out, leaving only the encryption with k'_2 , which employs a deterministic transform of a 64 b key, opening this to a moderately expensive offline bruteforce. The same approach

2. [2 points] A password generation scheme, which explicitly aims to generate passwords which are both long and employing the full 8-bit ASCII symbol set including numbers, letters, and uncommon characters. The password selection strategy works as follows:

- Split the 256 ASCII character set into 8 subsets of 32 contiguous characters. Note that the sets are non overlapping and cover all 256 ASCII characters.
- Iterating on the subsets draw a uniformly random character per subset. If the last subset is reached, start again from the first.

The strategy draws 12 characters with the method described above. State if the approach generates secure passwords against bruteforcing.

Each draw contributes 5 bits of information to the password; since there are 12 draws, the total information encoded in the password is 60 bits, which is still amenable to offline bruteforce attacks, with moderate computational resources.

Exercise 3 - Binary Vulnerabilities (6 points)

```
1. #include <stdio.h>
2. #include <stdbool.h>
3.
4. typedef struct {
5.     bool guardsAsleep;
6.     bool gateOpen;
7.     bool drawbridgeLowered;
8.     bool torchLit;
9.     char banner[64];
10. } State;
11.
12. int main(void) {
13.     State s = {0};
14.
15.     puts("=== The White Castle ===");
16.     printf("Name your banner: ");
17.     scanf("%63s", s.banner);
18.
19.     // I am lazy, I don't think the full implementation is worth the effort.
20.     // Who cares about all the boolean variables?
21.
22.     printf("Are the GUARDS asleep? (1=true, 0=false): ");
23.     scanf("%d", &s.guardsAsleep);
24.
25.     if (s.gateOpen && s.drawbridgeLowered && s.torchLit && s.guardsAsleep) {
26.         printf("You stride into the keep and plant the banner atop the White Castle!\n");
27.         printf(s.banner);
28.     } else if (!s.gateOpen && !s.drawbridgeLowered && !s.torchLit && !s.guardsAsleep) {
29.         puts("Everything is shut, dark, and alert. The White Castle remains impenetrable.");
30.     } else if (s.gateOpen && s.drawbridgeLowered && !s.guardsAsleep) {
31.         puts("The way is open, but the guards are awake – you are captured!");
32.     } else if ((s.gateOpen + s.drawbridgeLowered + s.torchLit + s.guardsAsleep) == 1) {
33.         puts("Only one omen favors you – not enough for a brave assault tonight.");
34.     } else {
35.         puts("Some conditions align, some do not. The White Castle bids you try another plan.");
36.     }
37.
38.     return 0;
39. }
```

The program is compiled for the **x86 architecture** (32 bit) and for an environment that adopts the usual **cdec1** calling convention. Furthermore, assume that **no compiler-level or OS-level mitigations** against the exploitation of memory corruption errors are present (unless specified otherwise).

Further Assumptions

Although C99 defines **bool** as an alias of **_Bool** with implementation-defined characteristics, you can definitely assume the common ABI where **bool** has the same layout characteristics (padding, size, etc.) as **uint8_t**.

[2 points] 1. Fill out the following table by reporting all the vulnerabilities you can spot in the code. If you believe there might be a vulnerability, specify for each vulnerability the **corresponding line/s of code** and **specify if it is possible to exploit it**.

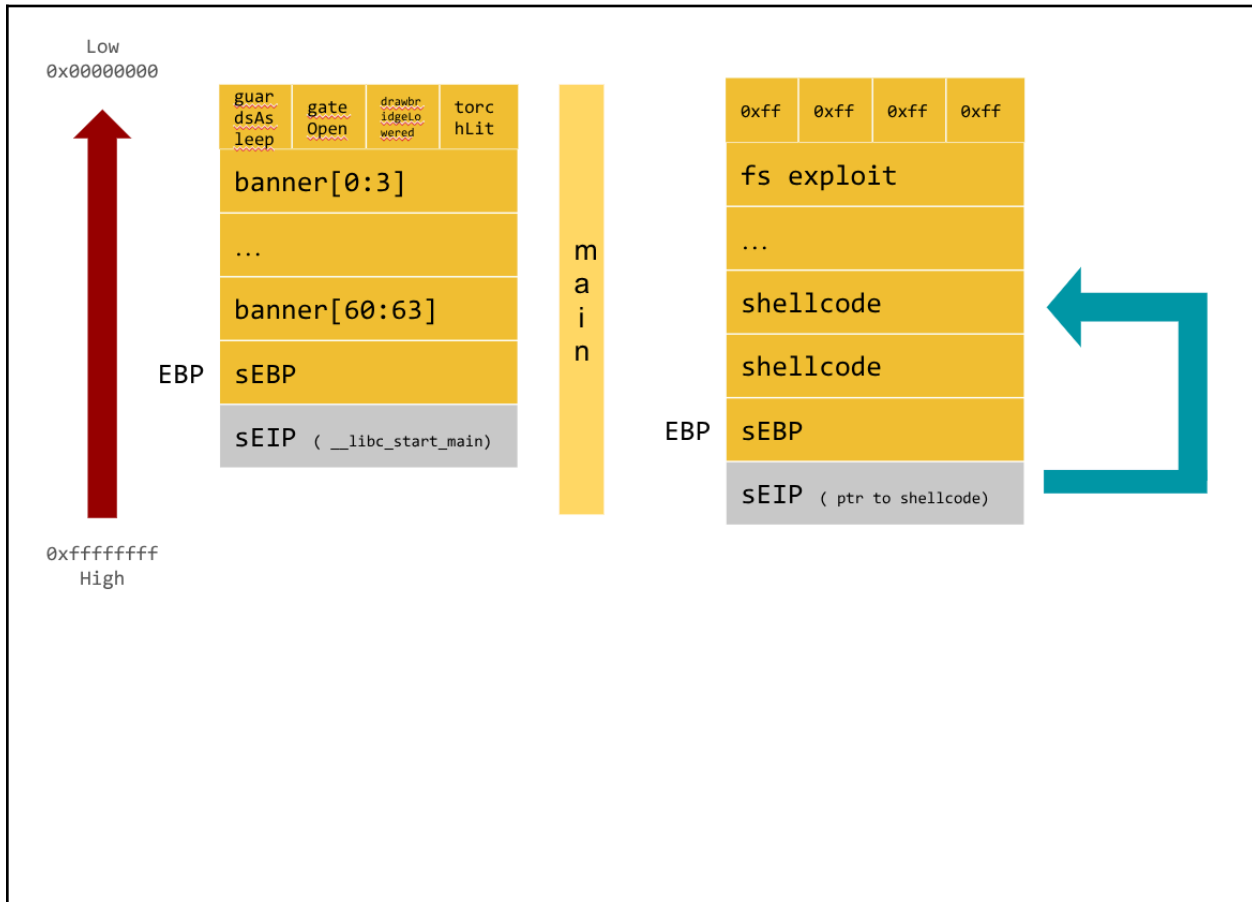
Vulnerability (e.g., format string, buffer overflow, ...)	Line	<u>Is it exploitable?</u>	
		YES	NO
[1 point] Buffer Overflow (alternatives highlighting wrong typing are accepted)	23	☒	☐
[1 point] Format String	27	☒	☐

[2 points] 2. Can you find a way not only to plant the banner on top of the White Castle, but also to take control of the program by executing **shellcode**? Assume the shellcode is **8 bytes long**: 0x50 0x50 0x50 0x50 0x50 0x50 0x50 0x50. List all assumptions and considerations that would make exploitation possible; **any missing assumption will cost points**. If exploitation is not possible, explain why. In your answer, be **exhaustive, concise, and straight to the point**.

You can exploit the type mismatch at line 23 to write a 32-bit value (%d) into an 8-bit type (bool), overflowing the variable. For example, writing 0xFFFFFFFF would set all four adjacent boolean variables (4×8 bits = 32 bits, i.e., the size of %d) to true. This unlocks the format string at line 27 (letting you plant the banner). The format string could then be used to overwrite the saved EIP (where to write) with the address of the shellcode (what to write). The shellcode could be placed in the same buffer as the banner, after the format string (placing it before would require carefully accounting for already printed characters and the shellcode bytes, which might interfere with the exploit— not the case of the provided shellcode). It could also be placed in environment variables or argv, under the usual assumption that you control the system where the program is running, as in challenge settings.

[2 points] 3. Show the effect of the identified vulnerabilities on memory. If you have found an exploit that enables execution of the shellcode described in point 2, illustrate the **stack layout both before and after** the execution of the most relevant line(s) for that exploit, including:

- The **direction of memory growth** and the **relative address order**;
- The **name of each relevant allocated variable** in the stack frame;
- The **contents of relevant registers**, such as **EBP** and **ESP**;
- The **stack frames of the current function** and any **callers**;
- The **contents of the caller's frame**, including saved return addresses and local variables.



Exercise 4 - Web Vulnerabilities (6 points)

MyPhotoGallery.com is a trending platform where users can upload photos, receive ratings, and comments. Each user has a public gallery that showcases their pictures with titles.

```
00 def set_csrf(session, ttl=600): # CSRF implementation reference
01     clear_expired_csrf(session)
02     token = str(get_random_uuid())
03     expiration = int(get_time()) + ttl
04     session["csrf_tokens"][token] = expiration
05     return token, expiration # Embedded in the final HTML form
06
07 def check_csrf(request, session):
08     clear_expired_csrf(session)
09     if "csrf_token" not in request.form.keys():
10         return False
11     token = request.form["csrf_token"]
12     if "csrf_expiration" not in request.form.keys():
13         return False
14     expiration = int(request.form["csrf_expiration"])
15     if token in session["csrf_tokens"].keys() or get_time() < expiration:
16         return True # CSRF is good
17     return False
18
19 @app.route('/post_photo', methods=['POST'])
20 def post_photo(request, session):
21     if not is_session_valid(session):
22         return redirect(url_for("login"))
23     user_id = session["user_id"]
24     if not check_csrf(request, session):
25         return render_template("home.html", error="Invalid CSRF token!")
26     if "title" not in request.form.keys():
27         return render_template("home.html", error="No title provided!")
28     title = request.form["title"]
29     if len(title) > 100:
30         return render_template("home.html", error="Title [" + title + "] too long!")
31     if "photo" not in request.files.keys():
32         return render_template("home.html", error="No photo provided!")
33     photo = request.files["photo"]
34     insert_q = "INSERT INTO photos (author_id, title) VALUES ('" + user_id + "', '"
35         + title + "');"
36     execute_query(insert_q)
37     path = "/photos/" + user_id + "/" + title
38     with open(path, "wb") as f:
39         f.write(photo.dataa())
40     return render_template("home.html", success="Photo posted successfully!")
```


The relational database schema used by the website is the following:

Table: **users**

user_id (Primary Key) (int) (auto increment)	username (String)	password (String)	email (String)
--	----------------------	----------------------	-------------------

Table: **photos**

author_id (Primary Key) (string) (Reference: users.user_id)	title (string)
---	-------------------

Assume the following:

- The session is correctly handled (i.e., you cannot forge arbitrary cookies).
- The function **is_session_valid** correctly checks if the user is logged in.
- The function **db_execute_query** executes queries without prepared statements.
- The function **db_execute_query_with_statements** executes queries with prepared statements.
- The function **render_template** does not perform any sanitization of the input.
- The **DBMS** does not support stacked queries (e.g., "SELECT ...; INSERT INTO ...;" fails if executed)
- The **DBMS** is case-sensitive; "Law" and "law" are two different users.

More details:

- The function **render_template** generates an HTML page from an .html file with optional parameters that are placed in the HTML code.
- The function **url_for** generates a URL for the provided page of the same domain. Optional query parameters of the URL can be specified as parameters.
- The function **redirect** redirects the user to the web page of the provided URL.
- The function **db_execute_query** returns a list of rows as a result, each of which has the fields specified by the **SELECT** statement. For instance, `result[4]["comment"]` is accessing the field "comment" of the 5th row.
- The function **get_random_uuid** returns a random UUID (e.g., `dd9bfc99-5c37-456a-b943-9a2e59b60324`).
- The function **get_time** returns the number of seconds since the Unix epoch. I.e., January 1st, 1970 UTC.
- The function **open** returns a file descriptor that can be used to read/write from/to the file. The function **open** has 2 parameters: the path where the file is or should be written on the filesystem, and the mode (read or write). For instance, **open("/tmp/test", "wb")** is going to create a file called "test" under "/tmp" and open it for binary writing. The file descriptor, typically named "f" or "fd", can be used to call the functions read and write on the file (e.g., `f.write(data)` or `data = f.read()`).

1. [1 point] For each SQL injection vulnerability you have identified, please provide the vulnerable line and the variable(s) that can be exploited for the injection.

Line	Variable(s)
34	title

2. [1 point] For each XSS vulnerability you have identified, please provide the vulnerable variable, the XSS category, and where you would fix it. Moreover, explain why

Variable	XSS category			Lines to sanitize. It can be between two lines (e.g., 21-22) or in one line.
	Stored	Reflected	DOM-based	
title	☐	☒	☐	28-29
	☐	☐	☐	
	☐	☐	☐	

3. [1 point] For each function, specify whether they are vulnerable to CSRF attacks. Moreover, explain why they are vulnerable.

Function	Vulnerable	Not vulnerable
post_photo	☒	☐

Motivation:

The CSRF token is completely random and unpredictable. Thus, correctly generated.
However, the check with the expiration time is performed with an or instead of an and. Therefore, any unexpired CSRF token will pass the check and make the function return true.

4. [1.5 points] While I was browsing the galleries, one alert popped up saying, “My photos should be in a museum!!!”. Can you reproduce the exploit? Explain all the steps you would take to reproduce the exploit, including the final code you would write.

The vulnerability is a reflected XSS vulnerability on the “title” variable.

If the length of the title is greater than 100 characters, the title will be returned without any filtering.

Thus, an exploit like

Title = “<SCRIPT>alert(“My photos should be in a museum!!!”</SCRIPT>AAAAAAAAAAAAAAAA...”

will do the work.

5. [1.5 points] All the photos in my photo gallery have been **replaced**. The titles are correct, but all the images just show cats. Can you reproduce the exploit without using the vulnerabilities identified in points 1, 2, and 3? Explain all the steps you would take to reproduce the exploit, including the final code you would write.

The vulnerability is a path traversal vulnerability.

The attacker inserted photos of cats with the title:

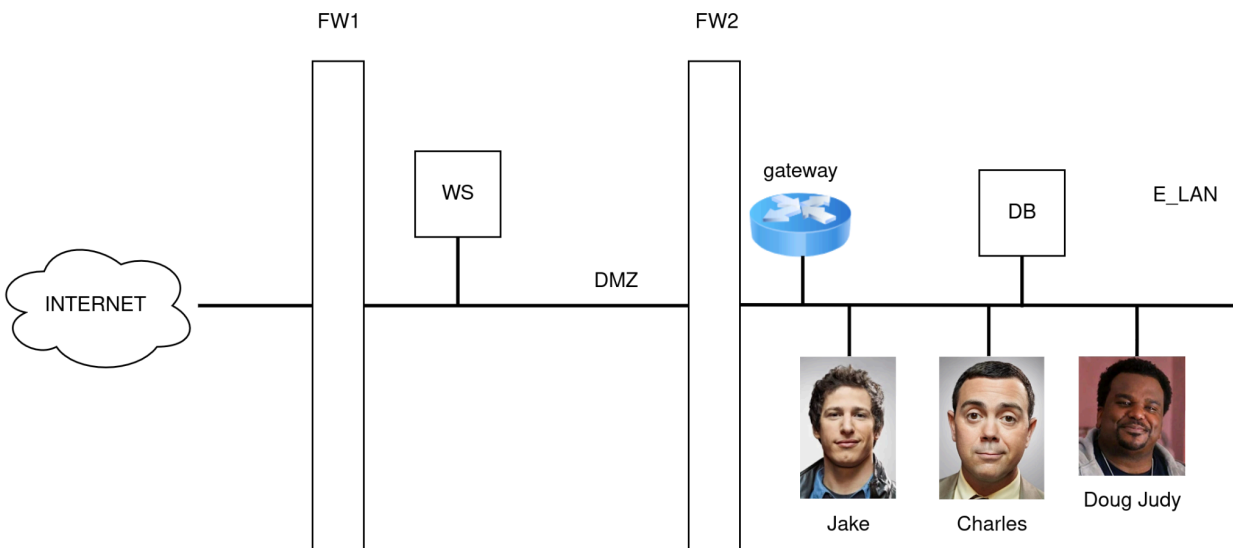
../<victim_id>/<existing_photo_title>

Exercise 5 - Network Security (6 points)

Precinct 99 in New York underwent a major revision of its IT infrastructure. With this new design, a DMZ zone was designed, and then two other internal LAN lines were created: one for the police officers and one for the backend.

In particular, they host their own web application on a dedicated server through which several operations are possible, such as booking an appointment, getting useful information, etc. In the precinct, they also need a dedicated database to trace criminals and their records. At last, there is a dedicated LAN line for all employees who should be able to reach the server in the DMZ, as well as access the Internet for work purposes (e.g., accessing their web-based email). Moreover, they are also capable of reaching out to the database for direct queries over it to retrieve useful information for their investigations.

Here is the proposed design.



1. [1 points] Write the firewall rules, assuming firewalls to be stateful packet filters (i.e. you can consider the response rules implicit). Assume all communications over **HTTP**.

Firewall	Src IP	Src PORT	Direction of the 1st packet	Dst IP	Dst PORT	Policy	Description
FW1 (example)	10.0.0.1 (example)	ANY	zone 1 -> zone 2	192.168.0.2 (example)	443	DENY	(example: the X server in zone 1 cannot contact the Y server)
<i>FW1</i>	<i>ALL</i>	<i>ANY</i>	<i>ANY</i>	<i>ALL</i>	<i>ANY</i>	<i>DENY</i>	<i>Default deny</i>
<i>FW1</i>	<i>ANY</i>	<i>ANY</i>	<i>Int -> DMZ</i>	<i>WS_IP</i>	<i>80 (HTTP)</i>	<i>ALLOW</i>	<i>WS publicly reachable</i>
<i>FW2</i>	<i>ALL</i>	<i>ANY</i>	<i>ANY</i>	<i>ALL</i>	<i>ANY</i>	<i>DENY</i>	<i>Default deny</i>

<i>FW2</i>	<i>ANY</i>	<i>ANY</i>	<i>E_LAN → DMZ</i>	<i>ANY</i>	<i>ANY</i>	<i>ALLOW</i>	<i>Employees reaching internet</i>
<i>FW1</i>	<i>ANY</i>	<i>ANY</i>	<i>DMZ → Internet</i>	<i>ANY</i>	<i>ANY</i>	<i>ALLOW</i>	<i>Employees reaching internet</i>
<i>FW2</i>	<i>ANY</i>	<i>ANY</i>	<i>E_LAN -> DMZ</i>	<i>WS_IP</i>	<i>CUST</i>	<i>ALLOW</i>	<i>Employees reaching WS</i>

2. [2 points] Despite all the precautions, Doug Judy, the friend/arch-rival of Detective Jake Peralta, managed to infiltrate the precinct itself, obtain control of one of the officers' PCs connected to the E_LAN. The goal of Doug Judy is to perform a man-in-the-middle attack to redirect traffic and steal/tamper with the data transmitted by Jake, who is located in the same E-LAN.

What attacks can he carry out among the ones reported in the table? Select the correct attack, if any.

For this question, assume all communications run over **HTTP**, and all devices in each LAN network have static **IPs** assigned. **Syn cookies** are enabled. **Domain names** are already resolved, cached on all hosts in the network, and cannot be updated.

☒ ARP spoofing	☐ DNS cache poisoning
☐ DHCP poisoning	☐ No attack can be done

Describe the **assumptions** and the steps Doug Judy should perform to put in place the selected attack. N.B. A general description of the attack, purely from theory, will not be considered a valid answer.

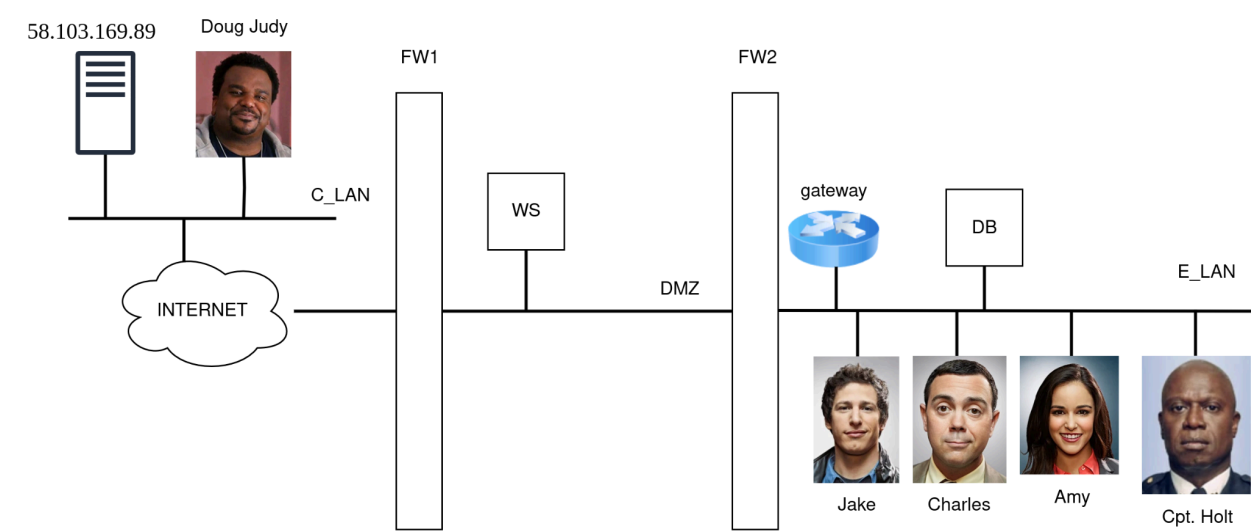
ASSUMPTIONS:

MAC addresses are not cached
Same LAN

ATTACK DESCRIPTION:

Doug Judy can impersonate the gateway through which all traffic pass through before going to the firewall. In this way he can sniff/tamper with all the traffic and specifically the packets transmitted by Jake.

3. [3 points] To avoid being caught, Doug Judy moves to a remote “secret” location, shown in the figure below. During an investigation, Amy Santiago, who is connected to the E_LAN, connects to an external server (IP 58.103.169.89), which contains the historical DB of old unsolved cases. Within this network, she finds a suspicious IP address (IP 58.103.169.101) that is associated with Doug Judy's operations. Doug Judy's goal is to filter network traffic related to his cases.



What attacks can he carry out among the ones reported in the table? Select the correct attack, if any.

Assume that connections run over **HTTP**, with **standard TCP**; also, **domain names** are already resolved, and the information about **IP and MAC addresses** is already cached on all hosts in the network, and cannot be updated.

☐ ARP spoofing	☐ DNS cache poisoning
☒ TCP session hijacking	☐ No attack can be done

Describe the **assumptions** and the steps Doug Judy should perform to put in place the selected attack.
N.B. A general description of the attack, purely from theory, will not be considered a valid answer.

ASSUMPTIONS: Same LAN and HTTP allows sniffing and readying packets. Spoofing server's IP to reply to Amy correctly, since packets were sniffed. ISN can be sniffed/guessed since same LAN of server

ATTACK DESCRIPTION:

Doug Judy can keep sniffing the LAN, reading all packets that pass through from the server to Amy and viceversa. Packets' content can be read since HTTP is in clear text.

When he reads a packet from Amy inquiring for one of Doug Judy's unresolved case:

1. DoS the server to block the valid communication and prevent the server from replying
2. Crafts a valid response spoofing its IP to be the server one and also using the valid, sniffed/guessed ISN number
3. Reply to Amy with the fake packet that does not reveal old cases related to him

Exercise 6 - Malware (6 points)

The systems of the Hyrule Kingdom have been infected by a mysterious curse. As the newly appointed Sage of Informatics, it is your duty to analyze this malware and uncover its secrets before the realm falls into darkness.

```
#include <stdint.h>
#include <stdlib.h>
#define THRESHOLD 0

int main() {

    uint64_t r1 = __rdtsc();

    __asm__("cpuid");

    uint64_t r2 = __rdtsc();

    if (r2 - r1 > THRESHOLD) {
        exit(-1);
    }

    encrypt_disk("C://");
}
```

Some information useful in analyzing the code:

- `__rdtsc()` returns the processor time stamp. The processor time stamp records the number of clock cycles since the last reset (e.g., the last call to `__rdtsc`).
- `__asm__` executes an assembly instruction.
- `cpuid` instruction returns information about the processor. When this instruction is executed under a hypervisor (e.g., a virtualized environment), its execution provokes a “vm-exit” transition, which increases the execution time of the `cpuid` instruction.

[4 point] 1. Please fill the following table showing the stealth techniques the malware correctly implements or tries to implement. Report the motivation only for the techniques you identify as implemented.

Technique	Is Present	Motivation (only if present)
Polymorphism	NO	[see slides for Polymorphism definition]
Anti-Virtualization	YES	The code checks whether the execution time of <code>cpuid</code> is unusually high. If it is, this indicates that a VM exit has occurred, suggesting the code is running inside a hypervisor.

Anti-Debugging	NO	[see slides for Anti-Debugging definition]
Event-Triggered Code	NO	[see slides for Event-Triggered Code]

[2 point] 2. Please identify which are the techniques badly implemented, and propose a solution to correctly implement them.

The THRESHOLD is currently set to 0. Since the CPU requires more than one clock cycle to execute `cpuid`, the value of `r2 - r1` will always be greater than 0.

Solution: define a reasonable THRESHOLD to distinguish between execution inside and outside hypervisors.